AssignmentThree

Classes & CRT

By: Eng. Islam Hamdy Mohamed

Guided By: Eng. Hassan Khaled

1. Snippets for the implemented code

```
package enum_pkg;
    typedef enum bit [1:0] {
        No_Parity   = 2'b00,
        ODD_Parity  = 2'b01,
        EVEN_Parity = 2'b10
    } Parity_t;
endpackage
```

Figure 1 Enum Package Definition (enum_pkg.sv)

```
package Uart_packet;

import enum_pkg::*;

class Uart_class;
    rand logic [7:0] data_in;
    rand enum_pkg::Parity_t parity_in;

    constraint Data_C {
        data_in dist { 8'h00 :/ 30, 8'hFF :/ 30, [8'h01:8'hFE] :/ 40};
        parity_in dist {No_Parity :/ 20, ODD_Parity :/ 40, EVEN_Parity :/ 40};
    }

    // Changed task to function void to prevent function-calling-task warning
    function void Print();
        $display("displaying the stimulus values : data_in = %0h, parity_in = %0s",
                  data_in, parity_in.name());
    endfunction

    function Uart_class generate_stimulus();
        Uart_class pkt = new();
        if (pkt.randomize()) begin
            pkt.Print();
            return pkt;
        end else begin
            $display("Randomization failed");
            return null;
        end
    endfunction
```

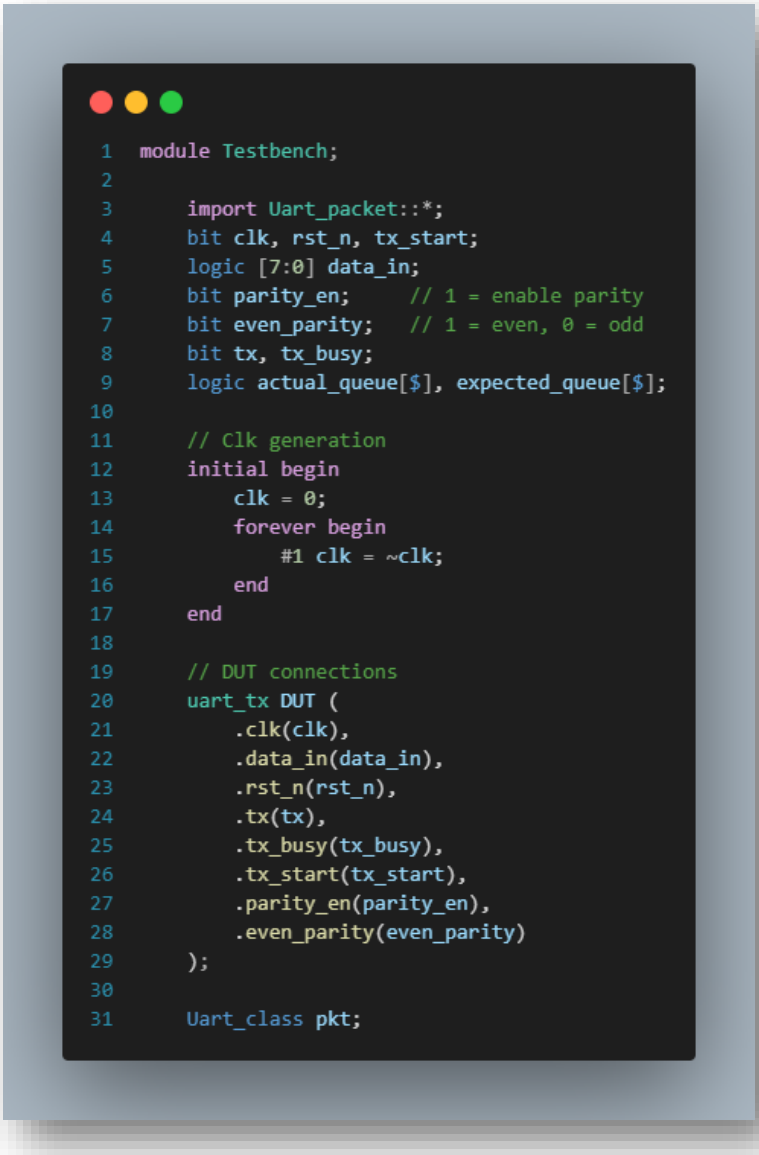
Figure 2 Uart_packet Class 1.1

```
task golden_model(ref logic expected_queue[$]);
    expected_queue.delete(); // Clear old contents

    expected_queue.push_back(1'b0);

    for (int i = 0; i < 8; i++) begin
        expected_queue.push_back(this.data_in[i]);
    end

    if (this.parity_in == enum_pkg::EVEN_Parity) begin
        expected_queue.push_back(~(^this.data_in)); // Even Parity[cite: 1]
    end else if (this.parity_in == enum_pkg::ODD_Parity) begin
        expected_queue.push_back(^this.data_in);   // Odd Parity[cite: 1]
    end

    expected_queue.push_back(1'b1);
endtask

task drive_stim(
    ref bit clk,
    ref bit tx_busy,
    ref bit tx_start,
    ref logic [7:0] tb_data_in,
    ref bit parity_en,
    ref bit even_parity
);
    if (tx_busy) begin
        $display("Notice: Transmitter is busy, waiting for it to finish...");
        wait (tx_busy == 1'b0);
    end

    // Drive stimulus using blocking assignments (=) for automatic ref variables
    @(posedge clk);
    tx_start    = 1'b1;
    tb_data_in  = this.data_in;
    parity_en   = (this.parity_in != enum_pkg::No_Parity);
    even_parity = (this.parity_in == enum_pkg::EVEN_Parity);

    @(posedge clk);
    tx_start    = 1'b0;
endtask
```

Figure 3 Uart_packet Class 1.2

```
    task collect_output(
        ref bit clk,
        ref bit tx_busy,
        ref bit tx,
        ref logic actual_queue[$]
    );
        int total_bits;
        total_bits = (this.parity_in == enum_pkg::No_Parity) ? 10 : 11;

        wait (tx_busy == 1'b1);
        wait (tx == 1'b0);
        repeat (total_bits) begin
            @(negedge clk); // Sampling at negedge guarantees stable data on tx pin
            actual_queue.push_back(tx);
        end
        wait (tx_busy == 1'b0);
    endtask

    task check_result(ref logic actual_queue[$], ref logic expected_queue[$]);
        bit match = 1;

        // Check if both queues have the same number of collected bits
        if (actual_queue.size() != expected_queue.size()) begin
            $display("[ERROR] Size mismatch! expected_size = %0d, actual_size = %0d",
                     expected_queue.size(), actual_queue.size());
            match = 0;
        end else begin
            // Compare each expected bit against the actual output bit
            foreach (expected_queue[i]) begin
                if (expected_queue[i] != actual_queue[i]) begin
                    $display("[MISMATCH] Bit [%0d]: Expected = %0b, Actual = %0b",
                             i, expected_queue[i], actual_queue[i]);
                    match = 0;
                end else begin
                    $display("[MATCH] Bit [%0d]: Expected = %0b, Actual = %0b",
                             i, expected_queue[i], actual_queue[i]);
                end
            end
        end

        // Report final comparison status for current packet
        if (match) begin
            $display("[TEST PASSED] Frame transmitted correctly!\n");
        end else begin
            $display("[TEST FAILED] Mismatch detected in transmission!\n");
        end

        // Flush queues to prepare for the next stimulus frame
        expected_queue.delete();
        actual_queue.delete();
    endtask

endclass
endpackage
```

Figure 1 Uart_packet Class 1.3



```
module Testbench;

    import Uart_packet::*;
    bit clk, rst_n, tx_start;
    logic [7:0] data_in;
    bit parity_en;     // 1 = enable parity
    bit even_parity;   // 1 = even, 0 = odd
    bit tx, tx_busy;
    logic actual_queue[$], expected_queue[$];

    // Clk generation
    initial begin
        clk = 0;
        forever begin
            #1 clk = ~clk;
        end
    end

    // DUT connections
    uart_tx DUT (
        .clk(clk),
        .data_in(data_in),
        .rst_n(rst_n),
        .tx(tx),
        .tx_busy(tx_busy),
        .tx_start(tx_start),
        .parity_en(parity_en),
        .even_parity(even_parity)
    );

    Uart_class pkt;
```

Figure 2 1.1 Top-Level Testbench Environment (Testbench.sv)

```
initial begin

        // Reset all
        rst_n = 0;
        tx_start = 0;

        // Check reset
        @(negedge clk);
        @(negedge clk);
        @(negedge clk);
        if (tx == 1'b1 && tx_busy == 1'b0) begin
            $display("=====> Reset button work correctly <=====");
        end else begin
            $display("=====> Reset button does not work correctly <=====");
        end

        // Check transition from IDLE to Start
        rst_n = 1'b1;
        tx_start = 1'b1;
        @(negedge clk);
        tx_start = 1'b0; // Clear tx_start so DUT can complete and clear tx_busy!

        if (tx == 1'b1 || tx_busy == 1'b1) begin
            $display("Transition from IDLE to START state DONE");
        end else begin
            $display("Transition from IDLE to START state FAILED");
        end

        // Wait for DUT to return to IDLE before starting the stimulus loop
        wait (tx_busy == 1'b0);

        // Sending Data
        repeat(100) begin
            pkt = new();
            pkt = pkt.generate_stimulus();

            pkt.golden_model(expected_queue);
            pkt.drive_stim(clk, tx_busy, tx_start, data_in, parity_en, even_parity);
            pkt.collect_output(clk, tx_busy, tx, actual_queue);
            pkt.check_result(actual_queue, expected_queue);
        end

        $display("=====> All Tests Completed <=====");
        $stop();

    end

endmodule
```

Figure 3 1.2 Top-Level Testbench Environment (Testbench.sv)

```
1  DATA: begin
2      tx <= shift_reg[0];              //FIX_ME::convert to 0 to fix ==> Done
3      shift_reg <= shift_reg >> 1;     //FIX_ME::convert to >> to fix ==> Done
4      bit_cnt <= bit_cnt + 1;
5      if (bit_cnt == 4'd7) begin
6          if (parity_en)
7              state <= PARITY;
8          else
9              state <= STOP;
10     end
11 end
12
```

Figure 4 RTL Correction for Data Bit Order Inversion (DATA State)

➢ **Description:**

1. **Original Bug:** The design was initially transmitting data MSB-first by driving tx <= shift_reg[7] and shifting left (shift_reg << 1), which violates the standard LSB-first UART transmission protocol.
2. **Applied Fix:** Updated line 2 to transmit from bit 0 (tx <= shift_reg[0]) and line 3 to shift right (shift_reg <= shift_reg >> 1), ensuring correct LSB-to-MSB sequence.

```
1  IDLE: begin
2      tx <= 1'b1;
3      tx_busy <= 1'b0;
4      if (tx_start) begin
5          shift_reg <= data_in;
6          bit_cnt <= 4'd0;
7          parity_bit <= (even_parity) ? ~(^data_in) : (^data_in);// Islam :: First bug parity_bit <= (even_parity) ? ~(^data_in) : (^data_in); ==> Done
8          tx_busy <= 1'b1;
9          state <= START;
10     end
11 end
12
```

Figure 5 RTL Correction for Parity Bit Logic Inversion (IDLE State)

➢ **Description:**

1. **Original Bug:** The conditional ternary logic inside the IDLE state inverted the parity bit calculation—generating odd parity when even_parity = 1 and even parity when even_parity = 0.
2. **Applied Fix:** Corrected line 7 to parity_bit <= (even_parity) ? ~(^data_in) : (^data_in);, accurately generating Even parity for mode 1 and Odd parity for mode 0.

❖ **Snippet for the waveform shows the different testcases**

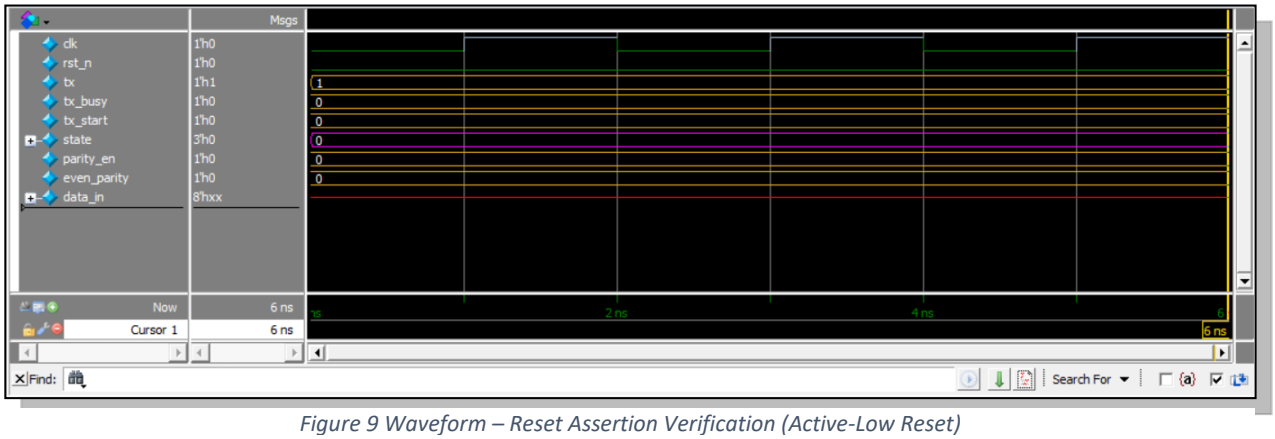


Figure 9 Waveform – Reset Assertion Verification (Active-Low Reset)

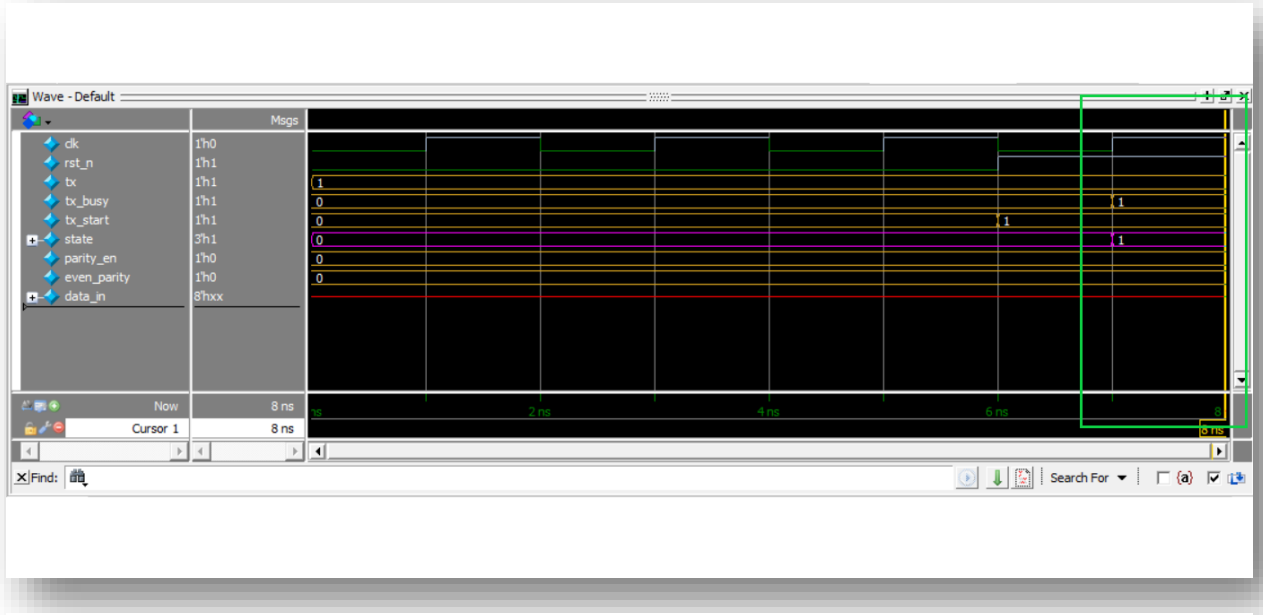


Figure 6 Waveform – FSM Transition from IDLE to START State

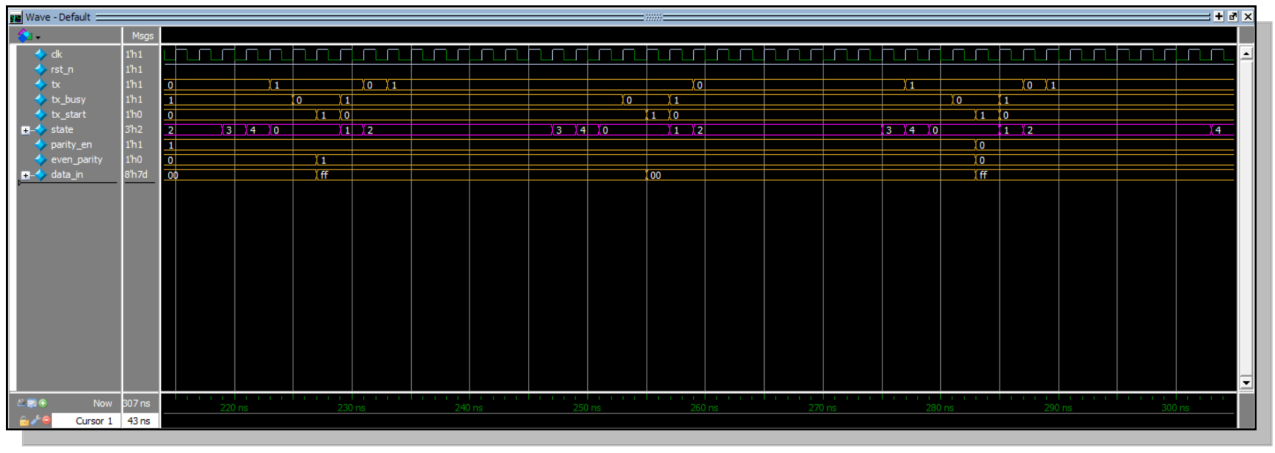


Figure 11 Waveform – Full FSM State Transitions Across Multiple Test Cases

❖ **Snippets for the logs show the all printed values.**

```
# =====> Reset button work correctly <=====
# ** Note: $stop    : Testbench.sv(40)
#    Time: 6 ns  Iteration: 1  Instance: /Testbench
```

Figure 12 Log(Check the reset button)

```
# Transtransition from IDLE to START state DONE
# ** Note: $stop    : Testbench.sv(51)
#    Time: 8 ns  Iteration: 1  Instance: /Testbench
```

Figure 7 Log(Transition From IDLE to Start)

```
displaying the stimulus values : data_in = 7d, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MISMATCH] Bit [1]: Expected = 1, Actual = 0
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 1, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 0, Actual = 0
[MISMATCH] Bit [9]: Expected = 0, Actual = 1
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST FAILED] Mismatch detected in transmission!

displaying the stimulus values : data_in = f8, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 0, Actual = 0
[MISMATCH] Bit [2]: Expected = 0, Actual = 1
[MISMATCH] Bit [3]: Expected = 0, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MISMATCH] Bit [7]: Expected = 1, Actual = 0
[MISMATCH] Bit [8]: Expected = 1, Actual = 0
[MISMATCH] Bit [9]: Expected = 1, Actual = 0
[MISMATCH] Bit [10]: Expected = 1, Actual = 0
[TEST FAILED] Mismatch detected in transmission!
```

Figure 14 Log shows a missmatch in the transmission

```
# displaying the stimulus values : data_in = ff, parity_in = EVEN_Parity
# [MATCH] Bit [0]: Expected = 0, Actual = 0
# [MATCH] Bit [1]: Expected = 1, Actual = 1
# [MATCH] Bit [2]: Expected = 1, Actual = 1
# [MATCH] Bit [3]: Expected = 1, Actual = 1
# [MATCH] Bit [4]: Expected = 1, Actual = 1
# [MATCH] Bit [5]: Expected = 1, Actual = 1
# [MATCH] Bit [6]: Expected = 1, Actual = 1
# [MATCH] Bit [7]: Expected = 1, Actual = 1
# [MATCH] Bit [8]: Expected = 1, Actual = 1
# [MATCH] Bit [9]: Expected = 1, Actual = 1
# [MATCH] Bit [10]: Expected = 1, Actual = 1
# [TEST PASSED] Frame transmitted correctly!
#
# displaying the stimulus values : data_in = 0, parity_in = EVEN_Parity
# [MATCH] Bit [0]: Expected = 0, Actual = 0
# [MATCH] Bit [1]: Expected = 0, Actual = 0
# [MATCH] Bit [2]: Expected = 0, Actual = 0
# [MATCH] Bit [3]: Expected = 0, Actual = 0
# [MATCH] Bit [4]: Expected = 0, Actual = 0
# [MATCH] Bit [5]: Expected = 0, Actual = 0
# [MATCH] Bit [6]: Expected = 0, Actual = 0
# [MATCH] Bit [7]: Expected = 0, Actual = 0
# [MATCH] Bit [8]: Expected = 0, Actual = 0
# [MATCH] Bit [9]: Expected = 1, Actual = 1
# [MATCH] Bit [10]: Expected = 1, Actual = 1
# [TEST PASSED] Frame transmitted correctly!
#
```

Figure 15 Log for testing the corners

```
displaying the stimulus values : data_in = 7d, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 1, Actual = 1
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 1, Actual = 1
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 0, Actual = 0
[MATCH] Bit [9]: Expected = 0, Actual = 0
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST PASSED] Frame transmitted correctly!

displaying the stimulus values : data_in = f8, parity_in = ODD_Parity
[MATCH] Bit [0]: Expected = 0, Actual = 0
[MATCH] Bit [1]: Expected = 0, Actual = 0
[MATCH] Bit [2]: Expected = 0, Actual = 0
[MATCH] Bit [3]: Expected = 0, Actual = 0
[MATCH] Bit [4]: Expected = 1, Actual = 1
[MATCH] Bit [5]: Expected = 1, Actual = 1
[MATCH] Bit [6]: Expected = 1, Actual = 1
[MATCH] Bit [7]: Expected = 1, Actual = 1
[MATCH] Bit [8]: Expected = 1, Actual = 1
[MATCH] Bit [9]: Expected = 1, Actual = 1
[MATCH] Bit [10]: Expected = 1, Actual = 1
[TEST PASSED] Frame transmitted correctly!
```

Figure 16 Log for different data with different parity bits

❖ **Snippet for Do file you have used for automating the process.**

```
vlib work
vlog Testbench.sv enum_pkg.sv Uart_packet.sv uart_tx.v
vsim -voptargs=+acc work.Testbench
do wave.do
run -all
#quit -sim
```

Figure 8 Do file used for automating the process

❖ Summary of Found Bugs & Fixes:

1. Data Bit Order Inversion (MSB-First Transmission)
    - Issue: The original RTL transmitted data bits MSB-first (shift_reg[7] with shift_reg << 1), violating standard LSB-first UART protocol rules.
    - Resolution: Updated the DATA state logic to send LSB-first using tx <= shift_reg[0] and shift_reg <= shift_reg >> 1.
2. Parity Bit Logic Inversion
    - Issue: The parity generation logic was inverted in the IDLE state, producing Odd parity when Even mode was selected and vice versa.
    - Resolution: Corrected the parity ternary statement to parity_bit <= (even_parity) ? ~(^data_in) : (^data_in).